Studio Nx

S1-13 Sentry 화면 오류 수집 — 함수 설명 합본 (백엔드 312, JS 313)

2026-10-21

목차

1. ai-notes/backend(fastAPI_Python)/S1-13 - Sentry 화면 오류 수집/312_api_client_config.md
2. ai-notes/javascript/S1-13 - Sentry 화면 오류 수집/313_core_sentry_init.md

백엔드 312 — 화면 설정 API (`GET /client-config`)

항목	내용
파일	`studionx/backend/app/api/v1/router.py`, `studionx/backend/app/schemas/client_config.py`, `studionx/backend/app/core/config.py`, `studionx/.env.example`
작업	S1-13 Sentry 화면 오류 수집: 화면이 시작할 때 읽는 공개 설정(브라우저용 Sentry DSN, 환경, 버전)을 내려 주는 API (PRD 00 §7)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 311 템포 변경점 연결, 다음 313 화면 오류 수집

1. 하는 일

화면(브라우저)이 오류를 Sentry로 보내려면 어느 Sentry 프로젝트로 보낼지(DSN)를 알아야 합니다. DSN을 화면 코드에 적어 두면 환경(개발, 배포)마다 파일을 고쳐야 하므로, 서버가 설정에서 읽어 내려 줍니다.

요청	응답
`GET /api/v1/client-config` (로그인 불필요)	`{ "sentry_dsn": "...", "environment": "dev", "release": "studionx@0.1.0" }`

```
studionx/.env 의 SENTRY_BROWSER_DSN ─▶ Settings.sentry_browser_dsn ─▶ GET /client-config ─▶ 화면(313)
  비어 있으면 화면은 오류를 보내지 않음 (기본값)
```

설정은 둘로 나뉩니다. 서버용 `SENTRY_DSN`은 서버가 낸 Python 오류를 받는 프로젝트이고, 새로 만든 `SENTRY_BROWSER_DSN`은 화면의 JS 오류를 받는 프로젝트입니다.

2. 언제 쓰나

쓰는 곳	하는 일
`core/sentry-init.js` (313)	화면에서 오류가 처음 났을 때 한 번 읽어 DSN을 얻습니다. 오류가 없으면 읽지 않습니다.
시험 `test_client_config.py`	응답 모양, 기본값, 로그인 없이 읽힘, 서버용 DSN이 새지 않음을 확인합니다.

3. 코드

3.1 응답 스키마

```
class ClientConfigResponse(BaseModel):
    """GET /client-config 응답."""

    sentry_dsn: str  # 비어 있으면 화면은 오류를 보내지 않음
    environment: str  # dev | prod (Sentry 의 environment)
    release: str  # 예: studionx@0.1.0
```

3.2 라우터

```
@api_router.get("/client-config", response_model=ClientConfigResponse, tags=["system"])
def client_config(request: Request) -> ClientConfigResponse:
    """화면이 시작할 때 읽는 공개 설정 (S1-13) — 브라우저용 Sentry DSN, 환경, 버전.

    로그인 없이 읽는다. 공개되어도 되는 값만 담는다 (schemas/client_config.py).
    """
    settings = get_settings()
    return ClientConfigResponse(
        sentry_dsn=settings.sentry_browser_dsn,
        environment=settings.app_env,
        release="studionx@" + request.app.version,
    )
```

3.3 설정 칸

```
# core/config.py 의 Settings
    # Sentry — 비어 있으면 비활성화
    sentry_dsn: str = ""                 # 서버(Python) 오류용
    sentry_traces_sample_rate: float = 0.1
    # 브라우저(화면)에서 난 JS 오류를 받는 Sentry 프로젝트의 DSN (S1-13)
    sentry_browser_dsn: str = ""         # 화면(JS) 오류용
```

```
# studionx/.env.example
SENTRY_BROWSER_DSN=
```

4. 왜 이렇게

선택	이유
로그인 없이 읽힙니다	오류는 로그인 화면(홈, 가입)에서도 나고, 로그인이 풀린 상태에서도 납니다. 인증이 필요하면 가장 보고 싶은 오류가 빠집니다.
DSN을 화면에 내려 줘도 됩니다	브라우저용 DSN은 화면 코드에 실려 누구나 볼 수 있게 설계된 공개 값입니다(Sentry 문서도 비밀로 다루지 않음). 이 값으로는 "이 프로젝트에 이벤트를 보내는 것"만 할 수 있고 읽을 수는 없습니다.
서버용 DSN은 절대 담지 않습니다	둘은 다른 값입니다. 시험(`test_server_dsn_never_leaks`)이 서버용 DSN이 응답 어디에도 없음을 확인합니다.
서버용과 브라우저용 프로젝트를 나눕니다	Python 오류와 JS 오류가 한 목록에 섞이면 원인을 가르기 어렵고, 브라우저 오류는 양이 많을 수 있어서 서버 오류가 묻힙니다. 같은 값을 넣어도 동작은 하므로 강제하지는 않습니다.
기본값은 빈 문자열입니다	설정하지 않으면 화면이 아무것도 보내지 않습니다. 개발 중이나 시험 중에 실수로 진짜 Sentry로 가는 일이 없습니다. `tests/conftest.py`가 시험에서는 이 값을 비웁니다.
`release`를 `studionx@0.1.0`으로 만듭니다	앱의 버전(`FastAPI(version=...)`)에서 읽어서 한 곳에서만 올립니다. Sentry가 오류를 버전별로 묶어 보여 줍니다.
화면이 매번 부르지 않고 오류가 날 때 한 번만 부릅니다	대부분의 페이지 로드에서 오류가 없으므로 요청이 늘지 않습니다.

5. 시험

`studionx/backend/tests/test_client_config.py`(4건)가 확인합니다.

- 연결: `/api/v1/client-config`에 `get`만 등록되어 있습니다.
- 기본값: 응답의 키가 `sentry_dsn`, `environment`, `release` 셋이고 DSN이 비어 있으며 `release`가 `studionx@0.1.0`입니다.
- 설정한 값: 브라우저용 DSN과 환경이 응답에 나옵니다. 로그인 헤더 없이도 200입니다.
- 서버용 DSN이 설정되어 있어도 응답 본문에 나타나지 않습니다.
- 백엔드 전체 시험 1109건이 통과합니다 (이전 1105건에 4건 추가). `ruff`도 통과합니다.

JavaScript 313 — 화면 오류 수집 (`core/sentry-init.js`)

항목	내용
파일	`studionx/frontend/static/js/core/sentry-init.js`, 그리고 첫 `import` 한 줄을 더한 페이지 진입 파일 8개 (`home.js`, `join.js`, `mypage.js`, `myproject.js`, `privacy.js`, `resetPassword.js`, `sequencer.js`, `userinfo.js`)
작업	S1-13 Sentry 화면 오류 수집: 화면에서 난 JS 오류를 Sentry로 보냅니다 (PRD 00 §7)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 312 화면 설정 API, 다음 없음 (S1-13 Sentry의 마지막 문서)

1. 하는 일

이 파일이 있기 전에는 템플릿(`TODO`만 있는 빈 함수)이었고, 어느 페이지도 불러오지 않았습니다. 화면에서 JS 오류가 나도 Sentry에는 아무것도 도착하지 않았습니다. 이제 이 파일을 불러오면 오류 처리기가 설치되고, 잡히지 않은 오류와 처리하지 않은 Promise 거절을 Sentry로 보냅니다.

```
페이지를 열면 (진입 파일의 첫 import) ─▶ sentry-init.js 가 실행되어 처리기(error, unhandledrejection)를 설치
  JS 오류가 남
    ├ judgeReport: 보낼 오류인가?  (ApiError, 무해한 알림, 확장 프로그램 오류, 이미 본 오류, 한도 초과는 안 보냄)
    ├ getConfig:   GET /client-config 로 DSN 을 한 번 받음 (DSN 이 비어 있으면 여기서 끝, 아무것도 안 보냄)
    ├ buildEvent:  주소의 query 와 hash, 글 속의 이메일과 토큰을 지우고 이벤트를 만듦
    └ 전송:        Sentry 의 envelope 주소로 POST (쿠키 없이, Referer 없이)
```

2. Sentry의 브라우저 스크립트를 쓰지 않고 직접 보냅니다

처음에는 Sentry의 브라우저 스크립트(SDK)를 CDN에서 불러오는 방식으로 설명드렸지만, 직접 보내는 방식으로 바꿨습니다.

비교	SDK를 불러오는 방식	직접 보내는 방식 (이 파일)
외부 코드	Sentry의 스크립트를 모든 페이지에 싣습니다.	없습니다. 이 파일 하나입니다.
보내는 내용	SDK가 정합니다(이동 경로, 입력 요소 정보 등이 함께 갈 수 있음).	이 파일이 정합니다. 보내는 항목이 코드에 모두 보입니다.
버전 관리	스크립트의 버전과 무결성 해시를 따로 관리해야 합니다.	해당 없음
화면 속도	스크립트를 내려받고 실행합니다(수십 KB).	영향이 없습니다.
기능	이동 경로 기록, 소스맵, 성능 측정, 자동 그룹화	오류 보내기만 합니다.

개인정보 처리방침(`privacy.html`)이 "비밀번호, 로그인 토큰, 쿠키는 보내기 전에 지웁니다"라고 약속하고 있어서, 무엇이 나가는지를 이 파일 하나에서 확인할 수 있는 쪽을 택했습니다. 대신 위 표의 기능 항목은 지원하지 않습니다. 나중에 필요해지면 SDK로 바꿀 수 있고, 처리기와 판단 규칙은 그대로 쓸 수 있습니다.

3. 언제 쓰나

쓰는 곳	하는 일
페이지 진입 파일 8개	첫 `import`로 이 파일을 불러옵니다. 그러면 처리기가 바로 설치됩니다.
직접 부르는 곳 (지금은 없음)	`captureException`으로 잡은 오류를 직접 보내거나, `reportMetric`으로 품질 지표를 보냅니다.
시험 `sentry_unit.mjs`, `ui_sentry.mjs`	순수 함수와, 실제 화면에서 가짜 Sentry로 가는 전송을 확인합니다.
확인용 `send_real_sentry.mjs`	실제 Sentry로 이벤트 한 건을 보내 `200`을 받는지 확인합니다.

4. 코드

4.1 순수 함수 (화면 없이 시험)

```
export function parseDsn(dsn) {
  if (typeof dsn !== 'string') return null;
  const m = /^(https?):\/\/([^:@\/]+)(?::[^@\/]*)?@([^\/]+)((?:\/[^\/]+)*)\/(\d+)\/?$/.exec(dsn.trim());
  if (!m) return null;
  return {
    key: m[2],
    host: m[3],
    projectId: m[5],
    url: m[1] + '://' + m[3] + m[4] + '/api/' + m[5] + '/envelope/?sentry_key=' + encodeURIComponent(m
[2]) + '&sentry_version=7',
  };
}
```

```
export function scrubUrl(url) {
  if (typeof url !== 'string') return '';
  const cut = url.search(/[?#]/);
  return cut >= 0 ? url.slice(0, cut) : url;
}
```

```
export function scrubText(text) {
  let s = String(text == null ? '' : text);
  s = s.replace(/(https?:\/\/[^\s?#]+)[?#]\S*/g, '$1');
  s = s.replace(/eyJ[A-Za-z0-9_-]{5,}\.[A-Za-z0-9_-]{5,}\.[A-Za-z0-9_-]*/g, '[jwt]');
  s = s.replace(/Bearer\s+[A-Za-z0-9._~+\/=-]+/gi, 'Bearer [Filtered]');
  s = s.replace(/[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}/g, '[email]');
  s = s.replace(/\b(token|password|passwd|secret|api[_-]?key|authorization|cookie|snx_refresh)(["']?\s*
[=:]\s*["']?)[^\s"'&,;]+/gi, '$1$2[Filtered]');
  return s;
}
```

```
export function parseStack(stack) {
  const frames = [];
  if (typeof stack !== 'string') return frames;
  stack.split('\n').forEach(function (line) {
    let m = /^\s*at\s+(?:(.*?)\s+\()?(.*?):(\d+):(\d+)\)?\s*$/.exec(line);
    if (!m) m = /^\s*(.*?)@(.*?):(\d+):(\d+)\s*$/.exec(line);
    if (!m) return;
```

```
    frames.push({
      function: m[1] || '?',
      filename: scrubUrl(m[2]),
      lineno: Number(m[3]),
      colno: Number(m[4]),
      in_app: m[2].indexOf('/static/js/') >= 0,
    });
  });
  return frames.reverse();
}
```

```
export function normalizeError(value) {
  if (value && typeof value === 'object' && (typeof value.message === 'string' || typeof value.stack === 'string')) {
    return { name: String(value.name || 'Error'), message: String(value.message || ''), stack: typeof value.stack === 'string' ? value.stack : '' };
  }
  if (typeof value === 'string') return { name: 'Error', message: value, stack: '' };
  let text = '';
  try { text = JSON.stringify(value); } catch (e) { text = String(value); }
  return { name: 'UnhandledRejection', message: 'Non-Error value: ' + text, stack: '' };
}
```

```
export function buildEvent(e, ctx) {
  const frames = parseStack(e.stack);
  const event = {
    event_id: newEventId(),
    timestamp: Date.now() / 1000,
    platform: 'javascript',
    level: ctx.level || 'error',
    environment: ctx.environment || 'dev',
    sdk: SDK,
    request: { url: scrubUrl(ctx.url), headers: { 'User-Agent': ctx.userAgent || '' } },
    tags: { page: pathOf(ctx.url) },
  };
  if (ctx.release) event.release = ctx.release;
  if (ctx.extra) event.extra = ctx.extra;
  const text = scrubText(e.message).slice(0, MAX_MESSAGE);
  if (ctx.isMessage) {
    event.message = text;
  } else {
    const value = { type: String(e.name || 'Error'), value: text, mechanism: { type: ctx.mechanism || 'generic', handled: false } };
    if (frames.length) value.stacktrace = { frames: frames };
    event.exception = { values: [value] };
  }
  return event;
}
```

```
export function buildEnvelope(event) {
  return JSON.stringify({ event_id: event.event_id, sent_at: new Date().toISOString() })
    + '\n' + JSON.stringify({ type: 'event' })
    + '\n' + JSON.stringify(event);
}
```

```
export function judgeReport(e, state) {
  if (e.name === 'ApiError') return { send: false, reason: 'api-error' };
  if (/ResizeObserver loop/i.test(e.message)) return { send: false, reason: 'noise' };
  if (e.message === 'Script error.' || e.message === 'Script error') return { send: false, reason: 'cros
s-origin' };
  if (/(chrome|moz|safari)-extension:\/\//.test(e.stack)) return { send: false, reason: 'extension' };
  if (state.count >= MAX_EVENTS) return { send: false, reason: 'limit' };
  if (state.seen[fingerprintOf(e)]) return { send: false, reason: 'duplicate' };
  return { send: true, reason: 'ok' };
}
```

4.2 보내기

```
function getConfig() {
  if (manualConfig) return Promise.resolve(manualConfig);
  if (!configPromise) {
    configPromise = fetch(CONFIG_URL, { credentials: 'omit' })
      .then(function (res) { return res.ok ? res.json() : null; })
      .then(function (c) {
        return c && c.sentry_dsn ? { dsn: c.sentry_dsn, environment: c.environment, release: c.release }
: null;
      })
      .catch(function () { configPromise = null; return null; });   // 서버에 닿지 못했으면 다음 오류 때
다시 묻기
  }
  return configPromise;
}
```

```
function send(e, ctxExtra) {
  return getConfig().then(function (config) {
    const dsn = config ? parseDsn(config.dsn) : null;
    if (!dsn) return { sent: false, reason: 'no-dsn' };
    const nav = typeof navigator !== 'undefined' ? navigator : null;
    const loc = typeof location !== 'undefined' ? location : null;
    const event = buildEvent(e, Object.assign({
      environment: config.environment,
      release: config.release,
      url: loc ? loc.origin + loc.pathname : '',
      userAgent: nav ? nav.userAgent : '',
    }, ctxExtra));
    return Promise.resolve(transport(dsn.url, buildEnvelope(event))).then(
      function () { return { sent: true, reason: 'ok', eventId: event.event_id }; },
      function () { return { sent: false, reason: 'send-failed' }; },
    );
  });
}
```

```
function defaultTransport(url, body) {
  return fetch(url, {
    method: 'POST',
    body: body,
    headers: { 'Content-Type': 'text/plain;charset=UTF-8' },   // 단순 요청이라 사전 확인(preflight) 없이
감
    credentials: 'omit',
```

```
    referrerPolicy: 'no-referrer',
    keepalive: true,
  });
}
```

```
export function captureException(error, mechanism) {
  try {
    const e = normalizeError(error);
    const verdict = judgeReport(e, state);
    if (!verdict.send) return Promise.resolve({ sent: false, reason: verdict.reason });
    state.count += 1;
    state.seen[fingerprintOf(e)] = true;
    return send(e, { mechanism: mechanism || 'manual' }).catch(function () { return { sent: false, reaso
n: 'error' }; });
  } catch (err) {
    return Promise.resolve({ sent: false, reason: 'error' });
  }
}
```

```
export function reportMetric(name, value) {
  try {
    const clean = String(name == null ? '' : name).replace(/[^A-Za-z0-9._-]/g, '').slice(0, 60);
    const n = Number(value);
    if (!clean || !isFinite(n)) return Promise.resolve({ sent: false, reason: 'invalid' });
    if (metricsSent[clean] || state.count >= MAX_EVENTS) return Promise.resolve({ sent: false, reason:
'limit' });
    metricsSent[clean] = true;
    state.count += 1;
    return send({ message: 'metric ' + clean }, { level: 'info', isMessage: true, extra: { value: n } })
      .catch(function () { return { sent: false, reason: 'error' }; });
  } catch (err) {
    return Promise.resolve({ sent: false, reason: 'error' });
  }
}
```

4.3 처리기와 설정

```
export function installHandlers(target) {
  const w = target || (typeof window !== 'undefined' ? window : null);
  if (!w || w.__snxErrorHandlers) return false;
  w.__snxErrorHandlers = true;
  w.addEventListener('error', function (ev) {
    if (!ev.error && !ev.message) return;
    captureException(ev.error || ev.message, 'onerror');
  });
  w.addEventListener('unhandledrejection', function (ev) {
    captureException(ev.reason, 'onunhandledrejection');
  });
  return true;
}
```

```
export function initSentry(options) {
  installHandlers();
  if (!options || !parseDsn(options.dsn)) return false;
```

```
  manualConfig = { dsn: options.dsn, environment: options.environment || 'dev', release: options.release
|| '' };
  return true;
}
```

5. 보내는 것과 보내지 않는 것

보냅니다	내용
오류	종류(`TypeError` 등), 문구(이메일과 토큰을 지운 뒤 500자까지), 호출 스택(파일 주소는 query를 지움)
위치	주소의 출처와 경로(`https://host/home.html`), 태그의 경로
환경	브라우저의 User-Agent, 환경(`dev`/`prod`), 버전(`studionx@0.1.0`)
방식	잡히지 않은 오류(`onerror`)인지 Promise 거절(`onunhandledrejection`)인지

보내지 않습니다	이유
주소의 `?` 뒤와 `#` 뒤	비밀번호 재설정 링크의 토큰이 query에 들어 있습니다 (`resetPassword.html?token=...`).
이메일, `token=`, `password:`, JWT, `Bearer` 값	문구에 들어갈 수 있어서 `scrubText`가 지웁니다.
쿠키, 인증 헤더	`credentials: 'omit'`이라 브라우저가 붙이지 않습니다.
Referer	`referrerPolicy: 'no-referrer'`라 지금 보던 주소가 헤더로 가지 않습니다.
사용자 번호, 이메일	사용자를 식별하는 값은 이벤트에 넣지 않습니다. (Sentry는 요청을 받으면서 IP를 알게 되며, 처리방침에 적혀 있습니다.)
서버가 일부러 준 오류(`ApiError`)	로그인이 풀리거나 입력이 잘못된 것은 오류가 아니라 정상 동작입니다. 서버의 500은 서버 쪽 Sentry가 이미 받습니다.
`ResizeObserver loop ...`, 내용 없는 `Script error.`	브라우저의 무해한 알림과, 다른 출처 스크립트가 내용을 숨긴 오류는 고칠 수 있는 정보가 없습니다.
확장 프로그램이 낸 오류	스택에 `chrome-extension://` 등이 있으면 우리 코드의 문제가 아닙니다.
같은 오류의 반복, 한 화면의 11번째부터	반복문 안에서 나는 오류가 Sentry를 쏟아붓지 않게 합니다 (`MAX_EVENTS` = 10).

6. 최신 문법으로 쓰면

```
// scrubUrl 과 judgeReport 를 최신 문법으로 쓴 모양
export const scrubUrl = (url) => (typeof url === 'string' ? url.split(/[?#]/)[0] : '');

export const judgeReport = (e, state) => {
  if (e.name === 'ApiError') return { send: false, reason: 'api-error' };
  if (/ResizeObserver loop/i.test(e.message)) return { send: false, reason: 'noise' };
  if (state.count >= MAX_EVENTS) return { send: false, reason: 'limit' };
  if (state.seen[fingerprintOf(e)]) return { send: false, reason: 'duplicate' };
  return { send: true, reason: 'ok' };
};

// 설정 얻기와 보내기
const config = await getConfig();
const dsn = config ? parseDsn(config.dsn) : null;
const event = buildEvent(e, { environment: config?.environment, release: config?.release, ...ctxExtra
});
```

전통 문법 (실제 코드)	최신 문법
`function (res) { return ... }`	화살표 함수 `(res) => ...`
`Object.assign({ ... }, ctxExtra)`	객체 전개 `{ ..., ...ctxExtra }`
`config ? parseDsn(config.dsn) : null`	`config?.dsn`과 `parseDsn(config?.dsn)`
`.then(...)`을 이어 붙임	`async`와 `await`
`loc ? loc.origin + loc.pathname : ''`	템플릿 리터럴과 옵셔널 체이닝을 함께 씀

7. 왜 이렇게

선택	이유
각 페이지 진입 파일의 첫 `import`로 둡니다	다른 파일을 불러오다 난 오류도 잡으려면 이 파일이 가장 먼저 실행되어야 합니다. 모듈은 `import` 순서대로 실행됩니다. 시험(`sentry_unit.mjs`)이 8개 파일 모두 첫 `import`가 이 파일인지 확인합니다.
이 파일은 다른 파일을 `import`하지 않습니다	`api.js` 같은 파일이 불러오다 실패해도 이 파일은 이미 실행되어 처리기가 설치되어 있어야 합니다. 시험이 `import` 줄이 없는지 확인합니다.
오류가 처음 났을 때만 설정을 받습니다	오류가 없는 대부분의 페이지 로드에서 요청이 하나도 늘지 않습니다. 설정을 못 받으면(서버에 닿지 못함) 다음 오류 때 다시 시도합니다.
DSN이 비어 있으면 아무것도 보내지 않습니다	설정하기 전이나 개발 중에는 꺼져 있는 것이 안전한 기본입니다.
같은 오류인지는 "종류, 문구, 난 자리(파일, 줄, 칸)"로 봅니다	문구만 같고 난 자리가 다르면 다른 오류입니다. 시험으로 확인했습니다.
오류를 보고하다 난 오류는 조용히 버립니다	보고 코드가 새 오류를 만들면 그 오류가 다시 보고되어 끝없이 반복될 수 있습니다. 모든 경로가 `try`, `catch`로 감싸져 있고 결과만 돌려줍니다.
단순 요청(`Content-Type: text/plain`)으로 보냅니다	사전 확인(preflight) 요청이 필요 없어서 요청이 하나입니다. Sentry가 이 방식을 받습니다.
`keepalive: true`	페이지를 닫거나 이동하는 중에 난 오류도 브라우저가 전송을 마칩니다.

8. 알려진 한계

한계	내용
Sentry 화면에서 어떻게 보이는지는 확인하지 못했습니다	실제 Sentry로 이벤트 한 건을 보내 `200`과 이벤트 번호를 돌려받는 것까지 확인했습니다 (개요 89 §2). Issues 목록에 어떤 제목과 스택으로 표시되는지는 Sentry에 들어가서 직접 보셔야 합니다.
소스맵이 없습니다	스택에는 실제 파일의 줄과 칸이 나옵니다. 이 프로젝트는 번들링 없이 원본 파일을 그대로 올리므로 소스맵이 필요하지 않습니다. 나중에 번들링하면 필요해집니다.
이동 경로(breadcrumb)와 성능 측정이 없습니다	오류가 나기 전에 사용자가 무엇을 눌렀는지는 기록하지 않습니다. 개인정보를 덜 보내는 쪽을 택했습니다.
오프라인에서 난 오류는 사라집니다	전송에 실패하면 다시 시도하지 않습니다.
서버 5xx는 화면에서 보내지 않습니다	서버 쪽 Sentry가 이미 받기 때문입니다. 서버가 응답하지 못하는 경우(네트워크 오류)는 둘 다 보지 못합니다.
빈 `catch`로 삼킨 오류는 보이지 않습니다	의도적으로 무시하는 곳(오디오 노드가 이미 끝났을 때 등)이 대부분이지만, 하나씩 확인한 것은 아닙니다.

9. 시험

- `docs/handoff/s1-13_tests/sentry_unit.mjs` (88건)는 브라우저 없이 확인합니다.
  - DSN: 키, 호스트, 프로젝트 번호, envelope 주소, 포트와 경로 접두어, 옛 형식, 잘못된 값 8가지가 `null`입니다.
  - 지우기: 주소의 query와 hash, 글 속의 이메일, `token=`, `password:`, `Bearer`, JWT, 주소 안의 query, 쿠키 이름 값을 지웁니다.
  - 스택: Chrome과 Firefox 두 형식, 오래된 호출부터의 순서, 파일 주소의 query 제거, `in_app` 표시, 이상한 입력입니다.
  - 이벤트: 번호가 32자리 16진수이고, 주소와 문구에 비밀이 남지 않으며, 헤더에 `User-Agent`만 있고, 문구는 500자까지이며, 지표는 `message`와 `info`입니다.
  - 판단: 보통 오류는 보내고, `ApiError`, `ResizeObserver`, `Script error.`, 확장 프로그램, 한도, 중복은 보내지 않습니다.
  - 보내기: 전송 함수를 바꿔 끼워, 같은 오류는 한 번만, 문구가 같아도 난 자리가 다르면 다른 오류, 한 화면에 10건까지, 전송이 실패해도 오류를 던지지 않음, DSN이 없으면 보내지 않음을 확인합니다.
  - 연결: 페이지 진입 파일 8개의 첫 `import`가 이 파일이고, 이 파일은 다른 파일을 `import`하지 않으며, 최신 문법(`?.`, `??`, `||=`)이 없습니다.
- `docs/handoff/s1-13_tests/ui_sentry.mjs` (27건)는 헤드리스 Chrome에서 실제 화면으로 확인합니다. 가짜 Sentry 서버와 앱 서버 둘(화면용 DSN이 가짜 Sentry를 가리키는 것과 비어 있는 것)을 시험이 직접 띄웁니다.
  - 서버가 화면에 DSN을 내려 주고, 페이지를 열기만 해서는 요청이 없습니다.
  - 주소에 `?token=URLSECRET#access_token=HASHSECRET`가 든 채로 오류를 내면, 보낸 3건 어디에도 그 값과 이메일, 토큰이 없습니다. 쿠키, 인증 헤더, Referer도 없습니다.
  - 잡히지 않은 오류(`onerror`)와 Promise 거절(`onunhandledrejection`)이 가고, `ApiError`와 `ResizeObserver` 알림은 가지 않으며, 같은 자리의 반복 오류는 한 번만 갑니다.
  - 오류 30개를 쏟아내도 10건에서 멈춥니다.
  - 로그인한 상태로 8개 페이지(`home`, `join`, `myPage`, `myProject`, `privacy`, `resetPassword`, `userInfo`, `studioNx`)를 열어 모두 처리기가 설치된 것을 확인합니다.
  - DSN이 비어 있는 서버에서는 오류가 나도 아무것도 보내지 않습니다.
- `docs/handoff/s1-13_tests/send_real_sentry.mjs`는 시험이 아니라 확인용입니다. 실제 Sentry로 이벤트 한 건을 보내 응답 `200`과 이벤트 번호를 받는지 봅니다 (개요 89 §2).
- 되돌려 보기: 페이지 하나(`home.js`)의 `import`를 지우거나, 이벤트 주소의 query 제거를 빼면 `sentry_unit.mjs`가 그 부분에서 실패하는 것을 확인했습니다.
- 시험 중 발견한 일: 처음에 "같은 오류는 한 번만"을 확인하려고 같은 문구의 `new Error('first')`를 두 줄에서 만들었더니 두 번 보내졌습니다. 난 자리가 달라서 다른 오류로 본 것이고 구현이 맞습니다. 시험을 한 오류 객체를 두 번 보내는 것으로 고치고, "문구가 같아도 난 자리가 다르면 다른 오류"를 별도로 확인합니다.